BART

Fine-Tuned

Steps:

1. Dataset Loading
2. Tokenizer
3. Preprocessing Function
4. Apply Preprocessing
5. Model Setup
6. Training Configuration
7. Data Collator and Trainer
8. Training Execution

1. Dataset Loading

Dataset: 7100 Indian Supreme Court judgments

Data Source: The dataset was created from 7.1k Indian Supreme Court case documents.

Data Splits: The data is divided into two splits: 'train' (7k docs) and 'test' (100 docs).

The dataset contains ‘document’ and ‘summary’ fields.

2. Tokenizer

Uses the fast tokenizer from HuggingFace for facebook/bart-large.

The tokenizer converts raw text (like "The accused was granted bail.") into a format that the model can understand — numbers.

1. Splits text into subword tokens (using Byte-Pair Encoding):

   "granted" → ["grant", "ed"]

2. Maps tokens to token IDs using the BART vocabulary:

   ["the", "accused", "was", "granted", "bail"] → [342, 8764, 1098, 2934, 1049]

3. Returns tensors:

   {'input_ids': [...], 'attention_mask': [...]}

3. Preprocessing Function

```
def preprocess_function(examples):
    inputs = [" ".join(doc) for doc in examples["document"]]
    targets = [" ".join(summ) for summ in examples["summary"]]
    model_inputs = tokenizer(inputs, max_length=1024, truncation=True, padding="longest")
    labels = tokenizer(targets, max_length=256, truncation=True, padding="longest")
    model_inputs["labels"] = labels["input_ids"]
    return model_inputs
```

3. Preprocessing Function

1. Join the tokens (if in token list form):

   inputs = ["The accused was granted bail."]

   targets = ["Bail was granted to the accused."]

2. Tokenize the document text:
   - Truncate to 1024 tokens max (BART's input limit)
   - Pad dynamically to the longest in the batch
3. Tokenize the summaries similarly, but with a shorter max length (256 tokens).
4. Assign target token IDs to the labels field, which BART uses for training (i.e., it learns to predict these summaries).

3. Preprocessing Function

Converts list-of-tokens to string for both inputs and summaries.

Tokenizes with:

- max_length=1024 for documents.
- max_length=256 for summaries.
- Uses "longest" padding for efficient batching.

Adds labels for supervised training.

3. Preprocessing Function

Final Output Looks Like:

```
{
  'input_ids': [...],          # tokenized document
  'attention_mask': [...],     # 1s and 0s for padding
  'labels': [...]              # tokenized summary (supervision signal)
}
```

4. Apply Preprocessing

Applies preprocessing and removes unnecessary columns.

5. Model Setup

Loads the BART model.

Enables gradient checkpointing to reduce GPU memory usage.

6. Training Configuration

```
args = TrainingArguments(
    output_dir="bart-legal",
    evaluation_strategy="no",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=1,
    gradient_accumulation_steps=8,
    weight_decay=0.01,
    num_train_epochs=3,
    fp16=True,
    save_total_limit=2,
    logging_steps=10,
    push_to_hub=False,
)
```

6. Training Configuration

Learning Rate: 2e-5
A small learning rate ensures stable training for large pre-trained models like BART.
2e-5 is commonly used for fine-tuning such models.

Batch Size: 1
Only 1 sample is processed per GPU step (due to memory limits).

Gradient Accumulation: 8
Gradients are accumulated for 8 steps before updating weights — effectively a batch size of 8.
Helps simulate a larger batch without exhausting GPU memory.

6. Training Configuration

Weight Decay: 0.01
Regularization to prevent overfitting by slightly penalizing large weights.

Epochs: 3
Trains the model for 3 full passes over the dataset.

Mixed Precision: fp16
Enables 16-bit floating point training for:

- Faster computation
- Lower memory usage

7. Data Collator and Trainer

```
data_collator = DataCollatorForSeq2Seq(tokenizer, model=model)

trainer = Trainer(
    model=model,
    args=args,
    train_dataset=tokenized_ds["train"],
    tokenizer=tokenizer,
    data_collator=data_collator,
)
```

7. Data Collator and Trainer

Automatically handles:

- Dynamic padding of inputs and labels
- Shifting decoder inputs for BART-style models

Ensures that each batch is correctly formatted for seq2seq training, even if input/output lengths vary.

8. Training Execution

Starts the fine-tuning process.

Example

Input (Document Text):
"The accused, after repeated warnings by the court, failed to appear on multiple hearings and did not submit required documentation to support his claim."

1. Tokenization:

```
inputs = tokenizer(text, return_tensors="pt", truncation=True, max_length=1024)
```

The raw sentence is tokenized into subwords and converted to token IDs.
BART adds <s> and </s> tokens to mark beginning/end.

```
Result: {
            'input_ids': [0, 133, 12893, 11, 4915, ..., 2],
            'attention_mask': [1, 1, 1, ..., 1]
        }
```

Example

2. Pass Through Fine-Tuned BART:

summary_ids = model.generate(**inputs, max_length=100, min_length=30)

Uses beam search to generate a coherent summary.

Predicts tokens one-by-one, conditioned on the input and prior outputs.

3. Decode the Output:

summary = tokenizer.decode(summary_ids[0], skip_special_tokens=True)

Final Output Summary:

"The accused failed to appear in court and did not provide supporting documents, despite repeated warnings."